

[Home](#) > [Blog](#) > [Data Engineering](#)

20 Kafka Interview Questions for Data Engineers

Prepare for your next data engineering interview with our extensive list of Kafka interview questions and answers.

Oct 23, 2024 · 15 min read

CONTENTS

Basic Kafka Interview Questions

1. What is Apache Kafka?
2. What are some of Kafka's features?
3. How do Partitions work in Kafka?
4. Why would you choose Kafka over other messaging services?
5. Explain all the APIs provided by Apache Kafka.

Intermediate Kafka Interview Questions

Advanced Kafka Interview Questions

Kafka Interview Questions for Data Engineers

Preparing For Your Kafka Interview



Training more people?

Get your team access to the full DataCamp for business platform.

For Business

For a bespoke solution [book a demo](#).

Understanding how data flows and leveraging its potential has become absolutely critical for modern organizations. Kafka is the leading platform for ingesting, storing, and processing real-time data streams in the modern application landscape.

In this post, we will outline some Kafka interview questions and explore the tool's crucial role in data processing. We will explore different levels of interview questions, from basic to advanced, and provide tips to help you prepare effectively. If you're new to Kafka, be sure to check out our [Introduction to Apache Kafka course](#).

Basic Kafka Interview Questions

These questions will test your basic understanding of Kafka as a tool for data engineering.

1. What is Apache Kafka?

Apache Kafka is a powerful, open-source distributed event-streaming platform. Originally developed by LinkedIn as a messaging queue, it has evolved into a tool for handling data streams across various scenarios.

Kafka's distributed system architecture allows horizontal scalability, enabling consumers to retrieve messages at their own pace and making it easy to add Kafka nodes (servers) to the cluster.

Kafka is designed to process large amounts of data quickly with low latency. Although it is written in Scala and Java, it supports a wide range of programming languages.



Apache Kafka acts as a distributed log collector, where log messages are stored as key-value pairs in an append-only log file for durable, long-term storage and retrieval. Unlike traditional message queues like RabbitMQ, which delete messages after they're consumed, Kafka retains messages for a configurable period, making it ideal for use cases that require data replay or event sourcing.

While RabbitMQ focuses on real-time message delivery without storing messages long-term, Kafka's retention policy supports more complex, data-driven applications.

Common use cases for Kafka include application tracking, log aggregation, and messaging, though it lacks traditional database features like querying and indexing. Its strength lies in handling real-time data streams, making it indispensable for distributed systems and real-time analytics.

2. What are some of Kafka's features?

Apache Kafka is an open-source distributed streaming platform widely used for building real-time data pipelines and streaming applications. It offers the following features:

1. High throughput

Apache Kafka is designed to handle high volumes of data with high throughput. It achieves this through its distributed architecture and efficient data storage and retrieval mechanisms. Kafka can process millions of messages per second, making it suitable for large-scale data streaming applications.

2. Distributed architecture

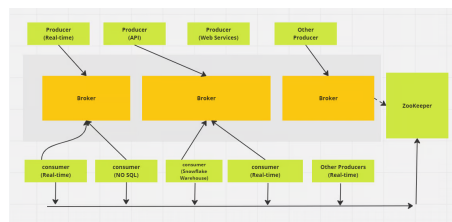
Apache Kafka has a cluster-centric architecture and inherently supports message [partitioning](#) across Kafka servers. This design also enables distributed consumption across a cluster of consumer machines, all while preserving the order of messages within each partition. Additionally, a Kafka cluster can scale elastically and transparently, without requiring any downtime.

3. Supporting various clients

Apache Kafka supports the integration of clients from different platforms, such as .NET, JAVA, PHP, and Python.

4. Real-Time messages

Kafka produces real-time messages that should be visible to consumers; this is important for complex event processing systems.



3. How do Partitions work in Kafka?

In Kafka, a topic serves as a storage space where all messages from producers are kept. Typically, related data is stored in separate topics. For instance, a topic named



Ashish Sharma

Seasoned Data, Analytics & AI Leader | Driving Innovation & Excellence

TOPICS

Data Engineering

Career Services



The Top 39 Data Engineering Interview Questions and Answers in 2025



Top 20 GCP Interview Questions: A Guide for All Skill Levels



Top 51 Data Architect Interview Questions and How To Answer Them



Top 24 Hadoop Interview Questions and Answers for 2025

DataCamp and our partners use cookies to improve your learning experience, offer content relevant to your interests and show more relevant advertisements. You can change your mind at any time ([learn more & configure](#)).

Accept

"transactions" would store details of user purchases on an e-commerce site, while a topic called "customers" would hold customer information.

Topics are divided into partitions. By default, a topic has one partition, but you can configure it to have more. Messages are distributed across these partitions, with each partition having its own offset and being stored on a different server in the Kafka cluster.

For example, if a topic has three partitions across three brokers, and a producer sends 15 messages, the messages are distributed in sequence:

- Record 1 goes to Partition 0
- Record 2 goes to Partition 1
- Record 3 goes to Partition 2

Then the cycle repeats, with Record 4 going back to Partition 0, and so on.

4. Why would you choose Kafka over other messaging services?

Choosing Kafka over other messaging services often comes down to its unique strengths, especially for use cases that need high throughput and real-time data processing. Here's why Kafka stands out:

- **High throughput and scalability:** Kafka can handle large volumes of data efficiently. Its architecture supports horizontal scaling, allowing you to add more brokers and partitions to manage increasing data loads without sacrificing performance.
- **Real-time processing:** It's excellent for real-time data streaming, making it perfect for use cases like activity tracking and operational monitoring. For example, LinkedIn created Kafka to handle its activity tracking pipeline, allowing it to publish real-time feeds of user interactions like clicks and likes.
- **Message replay:** Kafka allows consumers to replay messages, which is helpful if a consumer encounters an error or becomes overloaded. This ensures no data is lost—consumers can recover and replay missed messages to maintain data integrity.
- **Durability and fault tolerance:** The tool replicates data across multiple brokers, ensuring reliability even if some brokers fail. This fault-tolerant design keeps data accessible, providing security for critical operations.

5. Explain all the APIs provided by Apache Kafka.

An API (Application Programming Interface) enables communication between different services, microservices, and systems. Kafka provides a set of APIs designed to build event streaming platforms and interact with its messaging system.

The core Kafka APIs are:

- **Producer API:** Used to send real-time data to Kafka topics. Producers determine which partition within a topic each message should go to, with callbacks handling the success or failure of send operations.
- **Consumer API:** Allows reading real-time data from Kafka topics. Consumers can be part of a consumer group for load balancing and parallel processing, subscribing to topics and continuously polling Kafka for new messages.
- **Streams API:** Supports building real-time applications that transform, aggregate, and analyze data within Kafka. It offers a high-level DSL (Domain Specific Language) to define complex stream processing logic.
- **Kafka Connect API:** Used to create and run reusable connectors for importing and exporting data between Kafka and external systems.
- **Admin API:** Provides tools for managing and configuring Kafka topics, brokers, and other resources to ensure smooth operation and scalability.

These APIs enable seamless data production, consumption, processing, and management within Kafka.

Intermediate Kafka Interview Questions

These questions are aimed at more experienced practitioners who already have a firm understanding of Kafka's uses.

6. What are the implications of increasing the number of partitions in a Kafka topic?

Increasing the number of partitions in a Kafka topic can improve concurrency and throughput by allowing more consumers to read in parallel. However, it also introduces certain challenges:

- **Increased cluster overhead:** More partitions consume additional cluster resources, leading to higher network traffic for replication and increased storage requirements.
- **Potential data imbalance:** As partitions increase, data may not be evenly distributed, potentially causing some partitions to be overloaded while others remain underutilized.
- **More complex consumer group management:** With more partitions, managing consumer group assignments and tracking offsets becomes more complicated.
- **Longer rebalancing times:** When consumers join or leave, rebalancing partitions across the group can take longer, affecting overall system responsiveness.

7. Explain the four components of Kafka architecture

Kafka is a distributed architecture made up of several key components:

Broker nodes

Kafka's distributed architecture comprises several key components, one of which is the **broker node**. Brokers handle the heavy lifting of input/output operations and manage the durable storage of messages. They receive data from producers, store it, and make it available to consumers. Each broker is part of a Kafka cluster and has a unique ID to help with coordination.

In older versions of Kafka, the cluster is managed using **ZooKeeper**, which ensures proper coordination between brokers and tracks metadata such as partition locations (though Kafka is moving toward an internal KRaft mode for this). Kafka brokers are highly scalable and capable of handling large volumes of read and write requests for messages across distributed systems.

ZooKeeper nodes

ZooKeeper plays a crucial role in Kafka by managing broker registration and electing the Kafka controller, which handles administrative operations for the cluster. ZooKeeper operates as a cluster itself, called an ensemble, where multiple processes work together to ensure only one broker at a time is assigned as the controller.

If the current controller fails, ZooKeeper quickly elects a new broker to take over. Although ZooKeeper has been an essential part of Kafka's architecture, **Kafka is transitioning to a**

new KRaft mode that removes the need for ZooKeeper. ZooKeeper is an independent open-source project and not a native Kafka component.

Producers

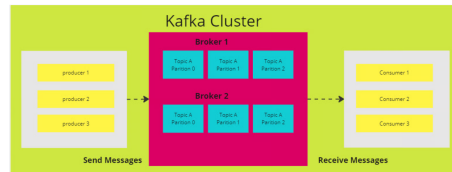
A Kafka producer is a client application that serves as a data source for Kafka, publishing records to one or more Kafka topics via persistent TCP connections to brokers.

Multiple producers can send records to the same topic concurrently. Kafka topics are append-only, meaning that producers can write new data, but neither producers nor consumers can modify or delete existing records, which ensures data immutability.

Consumers

A Kafka consumer is a client application that subscribes to one or more Kafka topics to consume streams of records. Consumers typically work in consumer groups, where the load of reading and processing records is distributed across multiple consumers.

Each consumer tracks its progress through the stream by maintaining offsets, ensuring that no data is processed twice and unprocessed records are not lost. Kafka consumers act as the final step in the data pipeline, where records are processed or forwarded to downstream systems.



Architecture diagram of Apache Kafka (Image by Author)

8. What is the primary purpose of log compaction in Kafka? How does log compaction impact the performance of Kafka consumers?

The main goal of log compaction in Kafka is to retain the most recent value for each unique key in a topic's log, ensuring that the latest state of the data is preserved and reducing storage usage. This allows consumers to access the current value more efficiently without having to process older duplicates.

Unlike Kafka's traditional retention policy, which deletes messages after a certain time period, log compaction deletes older records only for each key, retaining the latest value for that key. This feature helps ensure consumers always have access to the current state while maintaining a compact log for better storage efficiency and faster lookups.

9. What is the difference between Partitions and Replicas in a Kafka cluster?

Partitions and replicas are key components of Kafka's architecture, ensuring both performance and fault tolerance. **Partitions** increase throughput by allowing a topic to be split into multiple parts, enabling consumers to read from different partitions in parallel, which improves Kafka's scalability and efficiency.

Replicas, on the other hand, provide redundancy by creating copies of partitions across multiple brokers. This ensures fault tolerance because, in the event of a leader broker failure (the broker managing read and write operations for a partition), one of the follower replicas can be promoted to take over as the new leader.

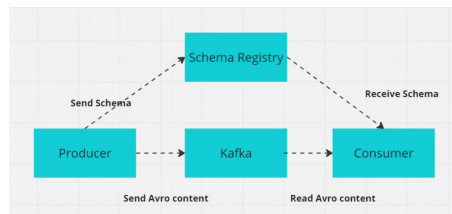
Kafka maintains multiple replicas of each partition to ensure high availability and data durability, minimizing the risk of data loss during failures, though full durability depends on replication settings.

10. What is a schema in Kafka, and why is it important for distributed systems?

In Kafka, a schema defines the structure and format of data, such as fields like CustomerID (integer), CustomerName (string), and Designation (string).

In distributed systems like Kafka, producers and consumers must agree on the data format to avoid errors when exchanging data. The Kafka Schema Registry manages and enforces these schemas, ensuring that both producers and consumers use compatible versions.

The Schema Registry stores schemas (commonly in formats like Avro, Protobuf, or JSON Schema) and supports schema evolution, allowing data formats to change without breaking existing consumers. This ensures smooth data exchange and system reliability as schemas evolve.



How Schema registry works with producer/consumer (Image by Author)

Advanced Kafka Interview Questions

For more Kafka advanced users, the interview questions you face are likely to be more technical in nature.

11. How does Kafka ensure message loss prevention?

Kafka employs several mechanisms to prevent message loss and ensure reliable message delivery:

1. **Manual offset commit:** Consumers can disable automatic offset commits and manually commit offsets after successfully processing messages to avoid losing unprocessed messages in case of a consumer crash.
2. **Producer acknowledgments (`acks=all`):** Setting `acks=all` on the producer ensures that messages are acknowledged by all in-sync replicas before being considered successfully written, reducing the risk of message loss in case of broker failure.
3. **Replication (`min.insync.replicas` and `replication.factor`):** Kafka replicates each partition's data across multiple brokers. A replication factor greater than 1 ensures fault tolerance, allowing data to remain available even if a broker fails. The `min.insync.replicas` setting ensures that a minimum number of replicas acknowledge a message before it is written, ensuring better data durability. For example, with a replication factor of 3, if one broker goes down, the data remains available on the other two brokers.
4. **Producer retries:** Configuring retries to a high value ensures that producers will retry sending messages if a transient failure occurs. Combined with careful configuration, like `max.in.flight.requests.per.connection=1`, this reduces the chance of message reordering and ensures that messages eventually get delivered without being lost.

12. How does Kafka differ from RabbitMq?

Kafka and RabbitMQ are two popular messaging systems and different in terms of architecture and usage. You can see how they compare in key areas:

1. Usage and design

Kafka is designed to handle large-scale data streams and real-time pipelines, optimized for high throughput and low latency. Its log-based architecture ensures durability and allows data to be reprocessed, making it ideal for use cases like event sourcing and stream processing.

RabbitMQ is a general-purpose message broker that supports complex routing and is typically used for messaging between microservices or distributing tasks among workers. It excels in environments where reliable message delivery, routing flexibility, and interaction between services are essential.

2. Architecture

Kafka categorizes messages into topics, which are divided into partitions. Each partition can be processed by multiple consumers, enabling parallel processing and scaling. Data is stored on disk with a configurable retention period, ensuring durability and allowing reprocessing of messages when needed.

RabbitMQ sends messages to queues, where they are consumed by one or more consumers. It ensures reliable delivery through message acknowledgments, retries, and dead-letter exchanges for handling failed messages. This architecture focuses on message integrity and flexibility in routing.

3. Performance and scalability

Kafka is built for horizontal scalability by adding more brokers and partitions, enabling it to handle millions of messages per second. Its architecture supports parallel processing and high throughput, making it ideal for large-scale data streaming.

RabbitMQ can scale, though not as efficiently as Kafka when dealing with very large volumes of data. It is suitable for moderate to high throughput scenarios but is not optimized for the extreme throughput Kafka can handle in large-scale streaming applications.

Summary: Both Kafka and RabbitMQ are powerful tools for messaging, but they excel in different use cases. Kafka is ideal for high-throughput, real-time data streaming, and event sourcing applications where large-scale, parallel processing is needed.

RabbitMQ is well-suited for reliable message delivery, task distribution in microservice architectures, and complex message routing. It excels in scenarios requiring loose coupling between services, asynchronous processing, and reliability.

13. How does Kafka help in developing microservice-based applications?

Apache Kafka is a valuable tool for building microservice architectures due to its capabilities in real-time data handling, event-driven communication, and reliable messaging. Here's how Kafka supports microservice development:

1. Event-driven architecture

- **Service decoupling:** Kafka allows services to communicate through events instead of direct calls, reducing dependencies between them. This decoupling enables services to produce and consume events independently, simplifying service evolution and maintenance.
- **Asynchronous processing:** Services can publish events and continue processing without waiting for other services to respond, improving system responsiveness and efficiency.

2. Scalability

- **Horizontal scalability:** Kafka can handle large data volumes by distributing the load across multiple brokers and partitions. Each partition can be processed by different consumer instances, allowing the system to scale horizontally.
- **Parallel consumption:** Multiple consumers can read from different partitions simultaneously, increasing throughput and performance.

3. Reliability and fault tolerance

- **Data replication:** Kafka replicates data across multiple brokers, ensuring high availability and fault tolerance. If one broker fails, another can take over with the replicated data. Kafka stores data on disk with configurable retention periods, ensuring events are not lost and can be reprocessed if necessary.

4. Data integration

- **Central data hub:** Kafka acts as a central hub for data flow within a microservice architecture, facilitating integration between various data sources and sinks. This simplifies maintaining data consistency across services.
- **Kafka Connect:** Kafka Connect provides connectors for integrating Kafka with numerous databases, key-value stores, search indexes, and other systems, streamlining data movement between microservices and external systems.

14. What is Kafka Zookeeper, and how does it work?

Apache ZooKeeper is an open-source service that helps coordinate and manage configuration data, synchronization, and group services in distributed systems. In older versions of Kafka, ZooKeeper played a critical role in managing metadata, leader election, and broker coordination.

However, Kafka has been transitioning to **KRaft mode** since version 2.8, which eliminates the need for ZooKeeper. **ZooKeeper is now considered obsolete and will be removed in the 4.0 release, which is expected in 2024.**

ZooKeeper stores data in a hierarchical structure of nodes called "znodes," where each znode can store metadata and have child znodes, similar to a file system. This structure is crucial for maintaining metadata related to Kafka brokers, topics, and partitions. ZooKeeper's quorum-based mechanism ensures consistency: a majority of nodes (quorum) must agree on any change, making ZooKeeper highly reliable and fault-tolerant.

In older versions of Kafka, ZooKeeper is used for several critical functions:

1. **Leader election:** ZooKeeper elects a controller node, which manages partition leader assignments and other cluster-wide tasks.
2. **Broker metadata management:** It maintains a centralized registry of active brokers and their status.
3. **Partition reassignment:** ZooKeeper helps manage the rebalancing of partitions across brokers during failures or cluster expansions.

ZooKeeper's ZAB protocol (ZooKeeper Atomic Broadcast) guarantees consistency across nodes, even in the face of network partitions, ensuring that Kafka remains fault-tolerant. This consistency and reliability make ZooKeeper crucial for managing the distributed nature of Kafka, though the future of Kafka's architecture will rely less on ZooKeeper as KRaft becomes the default mode.

15. How can you reduce disk usage in Kafka?

Kafka provides several ways to effectively reduce disk usage. Here are key strategies:

1. Adjust log retention settings

Modify the log retention policy to reduce the amount of time messages are retained on disk. This can be done by adjusting `retention.ms` (time-based retention) or `retention.bytes` (size-based retention) to limit how long or how much data Kafka stores before it deletes older messages.

2. Implement log compaction

Use log compaction to retain only the most recent message for each unique key, removing outdated or redundant data. This is particularly useful for scenarios where only the latest state is relevant, reducing disk space usage without losing important information.

3. Configure cleanup policies

Set effective cleanup policies based on your use case. You can configure both time-based (`retention.ms`) and size-based (`retention.bytes`) retention policies to delete older messages automatically and manage Kafka's disk footprint.

4. Compress messages

Enable message compression on the producer side using formats like **GZIP**, **Snappy**, or **LZ4**. Compression reduces message size, leading to lower disk space consumption. However, it's important to consider the trade-off between reduced disk usage and increased CPU overhead due to compression and decompression.

5. Adjust log segment size

Configure Kafka's **log segment size** (`segment.bytes`) to control how often log segments are rolled. Smaller segment sizes allow for more frequent log rolling, which can help in more efficient disk usage by enabling quicker deletion of older data.

6. Delete unnecessary topics or partitions

Periodically review and delete unused or unnecessary topics and partitions. This can free up disk space and help keep Kafka's disk usage under control.

Kafka Interview Questions for Data Engineers

If you're interviewing for a data engineer position, you may want to prepare for the following Kafka interview questions.

16. What are the differences between leader replica and follower replica in Kafka?

Leader replica

The leader replica handles all client read and write requests. It manages the partition's state and is the primary point of interaction for producers and consumers. If the leader fails, its role is transferred to one of the follower replicas to maintain availability.

Follower replica

The follower replica replicates data from the leader but does not directly handle client requests. Its role is to ensure fault tolerance by keeping an up-to-date copy of the partition's data. Starting from Kafka 2.4, under certain conditions, follower replicas can also handle read requests to help distribute the load and improve read throughput. If the leader fails, a follower is elected as the new leader to maintain data consistency and availability.

17. Why do we use clusters in Kafka, and what are their benefits?

A Kafka cluster is made up of multiple brokers that distribute data across several instances, allowing for scalability without downtime. These clusters are designed to minimize delays, and in case the primary cluster fails, other Kafka clusters can take over to maintain service continuity.

The architecture of a Kafka cluster includes Topics, Brokers, Producers, and Consumers. It efficiently manages data streams, making it ideal for big data applications and the development of data-driven applications.

18. How does Kafka ensure message ordering?

Kafka ensures message ordering in two main ways:

1. **Using message keys:** When a message is assigned a key, all messages with the same key are sent to the same partition. This ensures that messages with the same key are processed in the order they are received, as Kafka preserves ordering within each partition.
2. **Single-threaded consumer processing:** To maintain order, the consumer should process messages from a partition in a single thread. If multiple threads are used, set up separate in-memory queues for each partition to ensure that messages are processed in the correct order during concurrent handling.

19. What Do ISR and AR represent in Kafka? What does ISR expansion mean?

ISR (in-sync replicas)

ISR refers to the replicas that are fully synchronized with the leader replica. These replicas have the latest data and are considered reliable for both read and write operations.

AR (assigned replicas)

AR includes all replicas assigned to a partition, both in-sync and out-of-sync replicas. It represents the complete set of replicas for a partition.

ISR expansion

ISR expansion occurs when new replicas catch up with the leader and are added to the ISR list. This increases the number of up-to-date replicas, improving fault tolerance and reliability.

20. What is a zero-copy usage scenario in Kafka?

Kafka uses zero-copy to efficiently transfer large volumes of data between producers and consumers. It leverages the FileChannel.transferTo method to move data directly from the file system to network sockets without additional copying, improving performance and throughput.

This technique utilizes memory-mapped files (mmap) for reading index files, allowing data buffers to be shared between user space and kernel space, further reducing the need for extra data copying. This makes Kafka well-suited for handling large-scale real-time data streams efficiently.

Preparing For Your Kafka Interview

As you prepare for a Kafka-focused data engineering interview, it's crucial to have both a strong technical foundation and a clear understanding of how Kafka integrates into modern systems. Kafka is central to many real-time data platforms, so interviewers will expect you to know both the basics and how Kafka works in scalable, fault-tolerant environments.

Key areas to focus on

As we've seen across the Kafka interview questions we've covered here, there are several key areas you'll need to brush up on:

- **Architecture:** Be confident in discussing Kafka's architecture, including brokers, partitions, and the interaction between producers and consumers. You should also explain Kafka's message delivery guarantees—like "at-least-once" or "exactly-once" delivery—and how Kafka ensures message ordering through features like In-Sync Replicas (ISR).
- **Integration:** Understand how Kafka integrates with other systems using tools like Kafka Connect, which connects databases, key-value stores, and other systems. Interviewers may ask about Kafka's role in maintaining data consistency in microservices or distributed systems.
- **Monitoring and troubleshooting:** Familiarity with monitoring tools like Kafka Manager and Prometheus is important. You'll need to demonstrate your ability to manage and optimize Kafka clusters effectively.
- **Problem-solving:** Be prepared to design messaging solutions that solve real-world challenges, such as handling large data flows or ensuring message reliability. Interviewers may give you hypothetical scenarios where you'll need to troubleshoot or optimize Kafka performance.
- **Communication:** Kafka is complex, so you must be able to clearly explain your design decisions or troubleshooting steps, whether to technical teams or business stakeholders. Strong communication skills will set you apart.
- **Hands-on experience:** Highlight any practical experience, such as setting up Kafka clusters, managing data retention, or integrating Kafka with other tools. Practical knowledge is highly valued.
- **Continuous learning:** Stay updated with Kafka's latest features and best practices. Reading documentation or contributing to open-source projects shows your commitment to ongoing learning.

Interview tips

Here are some top tips for making sure you ace the interview:

- Research the company, industry, and your interviewer's background.
- Prepare for different interview rounds (technical, HR, phone, and in-person) with a tailored approach.
- Regularly review technical resources like:
 - [Apache Kafka for Beginners: A Comprehensive Guide](#)
 - [Introduction to Apache Kafka](#)
 - [The Top 35 Data Engineering Interview Questions and Answers in 2024](#)
 - [Kafka Partitions: Essential Concepts for Scalability and Performance](#)

By honing these skills and preparing effectively, you'll be well-equipped for Kafka-related interviews. Remember, interviewers are looking not only for technical expertise but also for clear communication and real-world problem-solving skills. Show your passion for data engineering throughout the interview process.



AUTHOR

Ashish Sharma

Ashish has 12 years of professional experience with a proven track record as a Solution Architect, on Microsoft Technology. He has successfully led teams in the execution of projects related to data governance, leading data analytics, reporting and insights, and visualization.

TOPICS

Data Engineering

Career Services

Related



BLOG

The Top 39 Data Engineering Interview Questions and...

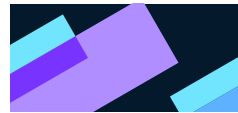
Ace your next interview with this compilation of data engineer interview questions and answers, helping you...



BLOG

Top 20 GCP Interview Questions: A Guide for All Skill Levels

Approach your GCP interview with confidence. Leverage expert tips and access practical interview questions and...



BLOG

Top 51 Data Architect Interview Questions and How To Answer...

Prepare to excel in your next data architect interview with this comprehensive guide, which includes top...

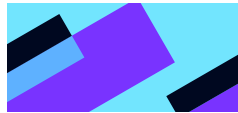


BLOG

Top 24 Hadoop Interview Questions and Answers for 2025

Discover the top Hadoop interview questions and answers to help you prepare for your next big data job interview. From...

Laiba Siddiqui • 15 min

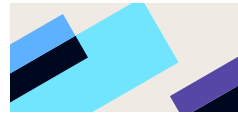


BLOG

Top 30 Cloud Computing Interview Questions and...

Explore key cloud computing interview questions and answers, from basic to advanced, to help you prepare for cloud...

Marie Fayard • 15 min



BLOG

28 Top Data Scientist Interview Questions For All Levels

Explore the top data science interview questions with answers for final-year students and professionals looking for jobs.

Abid Ali Awan • 15 min

[See More →](#)

Grow your data skills with DataCamp for Mobile

Make progress on the go with our mobile courses and daily 5-minute coding challenges.



LEARN

[Learn Python](#)

[Learn AI](#)

[Learn Power BI](#)

[Learn Data Engineering](#)

[Assessments](#)

[Career Tracks](#)

[Skill Tracks](#)

[Courses](#)

[Data Science Roadmap](#)

DATA COURSES

[Python Courses](#)

[R Courses](#)

[SQL Courses](#)

[Power BI Courses](#)

[Tableau Courses](#)

[Alteryx Courses](#)

[Azure Courses](#)

[AWS Courses](#)

[Google Sheets Courses](#)

[Excel Courses](#)

[AI Courses](#)

[Data Analysis Courses](#)

[Data Visualization Courses](#)

[Machine Learning Courses](#)

[Data Engineering Courses](#)

[Probability & Statistics Courses](#)

DATALAB

[Get Started](#)

[Pricing](#)

[Security](#)

[Documentation](#)

CERTIFICATION

[Certifications](#)

[Data Scientist](#)

[Data Analyst](#)

[Data Engineer](#)

[SQL Associate](#)

[Power BI Data Analyst](#)

[Tableau Certified Data Analyst](#)

[Azure Fundamentals](#)

[AI Fundamentals](#)

RESOURCES

[Resource Center](#)

[Upcoming Events](#)

[Blog](#)

[Code-Alongs](#)

[Tutorials](#)

[Docs](#)

[Open Source](#)

[RDocumentation](#)

[Book a Demo with DataCamp for Business](#)

[Data Portfolio](#)

PLANS

[Pricing](#)

[For Students](#)

[For Business](#)

[For Universities](#)

[Discounts, Promos & Sales](#)

[Expense DataCamp](#)

[DataCamp Donates](#)

FOR BUSINESS

[Business Pricing](#)

[Teams Plan](#)

[Data & AI Unlimited Plan](#)

[Customer Stories](#)

[Partner Program](#)

ABOUT

[About Us](#)

[Learner Stories](#)

[Careers](#)

[Become an Instructor](#)

[Press](#)

[Leadership](#)

[Contact Us](#)

[DataCamp Español](#)

[DataCamp Português](#)

[DataCamp Deutsch](#)

[DataCamp Français](#)

SUPPORT

[Help Center](#)

[Become an Affiliate](#)



[Privacy Policy](#) [Cookie Notice](#) [Do Not Sell My Personal Information](#) [Accessibility](#) [Security](#) [Terms of Use](#)

© 2025 DataCamp, Inc. All Rights Reserved.

